COS 487 / 587 Course Project Part 1: IR Rangers

Esther Greene
University of Southern Maine
Portland, Maine, USA
esther.greene@maine.edu

Courtney Jackson
University of Southern Maine
Portland, Maine, USA
courtney.jackson@maine.edu

Nathaniel Serrano
University of Southern Maine
Portland, Maine, USA
nathaniel.serrano@maine.edu

Abstract

...

Keywords

information retrieval, ...

1 Introduction

The volume of scientific literature makes finding relevant work increasingly difficult, both at the level of whole papers and of the specific passages that answer a question. This paper describes the first phase of our retrieval system for arXiv papers, which applies two traditional term-matching models, TF-IDF and BM25, to retrieving both papers and individual paragraphs. We describe our preprocessing of a noisy collection, tune BM25 per task, evaluate both models with significance testing, and analyze where term matching succeeds and where it fails, particularly on hard queries that paraphrase technical terminology. As we will detail, our findings show that BM25 significantly outperforms TF-IDF on both tasks.

The test collection consists of 34,308 arXiv papers published in August and September 2024. Each paper is provided both as raw HTML scraped from the ar5iv website and as a processed JSON file containing its identifier, title, abstract, and paragraphs. Because the JSON files were produced with rule-based extraction, they contain noise such as leftover LaTeX markup and merged words, which we discuss in Section 2.1. In total, the collection contains 2,275,574 paragraphs, which averages out to approximately 66 paragraphs per paper.

We address two tasks over this collection: *paper retrieval*, which returns papers identified by their arXiv ID (in YYMM.NNNNN format), and *paragraph retrieval*, which returns individual paragraphs. For each query, systems return their top 100 results. Three query sets are provided: Train (2,000 queries), Study (250 queries with relevance judgments, which we use for all analysis in this paper), and Test (250 queries whose judgments are withheld). Each set contains 30% easy, 50% medium, and 20% hard queries. Relevance is graded on a three-point scale: a document that fully addresses the query is relevant (2), one that provides useful information without fully addressing it is partially relevant (1), and any document not in the judgments is considered non-relevant.

2 Methodology

Our system processes both retrieval tasks with a single pipeline. Documents and queries are first cleaned and tokenized (Section 2.1); the processed text is used to build two collections, one of papers and one of paragraphs (Section 2.2), each with its own inverted index. Two ranking models, TF-IDF (Section 2.3) and BM25 (Section 2.4), score documents from these indexes; both use identical preprocessing and the same indexes, so differences between them reflect only their scoring functions. Section 2.5 describes our evaluation setup, and Section 2.6 the search interface.

Throughout, N denotes the number of documents in a collection, df_t the number of documents containing term t, $tf_{t,d}$ the number of occurrences of t in document d, and L_d and L_{avg} the length of d and the average document length, in tokens.

2.1 Preprocessing

Documents and queries pass through the same preprocessing pipeline, so that query terms match the terms stored in the index. All settings are read from a YAML configuration file, and a copy of the configuration is saved with the processed collections; queries are tokenized with that saved copy at search time. The final pipeline applies the following steps in order:

(1) **Text cleaning.** Inline mathematics delimited by dollar signs, remaining LaTeX commands (e.g., `\mathcal`), URLs, and numeric citation markers such as "[5, 6, 7]" are each replaced with a space.
(2) **Lowercasing.**
(3) **Tokenization.** Text is split into runs of letters and digits. Underscores act as separators, so a subscript such as `x_i` yields `x` and `i`.
(4) **Token filtering.** Tokens shorter than two characters and tokens consisting only of digits are discarded.
(5) **Stopword removal.** Stopwords are frequent function words, such as "the" and "and", that say little about a document's topic. We use NLTK's English list [1], extended with terms common in scientific writing: "et", "al", "fig", "figure", "table", "eq", and "section".
(6) **Stemming.** Remaining tokens are reduced to stems with the Snowball English stemmer [3], so that forms such as "model" and "models" match.

The cleaning and filtering steps were motivated by corpus statistics. With only lowercasing and stopword removal, the 50 most frequent tokens were dominated by mathematical notation rather than words: digits ("1", "2", "0"), single-letter variables ("n", "x", "k"), underscores from subscripts, and LaTeX commands such as "mathcal" and "mathbb". Adding these to the stopword list would also remove real words such as "text" and "prime", so we instead remove inline math as a whole. A lone dollar sign, such as in a price, would be paired with a math delimiter and the text between them deleted, so math spans are only removed from paragraphs with an even number of dollar signs. Only 1,458 paragraphs (0.06%) contain an unpaired dollar sign. They keep their math spans, although LaTeX command removal and the token filters still remove most of the resulting tokens. We evaluate the effects of our choice to employ inline math removal and stemming on retrieval in Section 4.

Some extraction noise is not addressed by preprocessing. Adjacent words are occasionally merged (e.g., “andenthusiasts”, “inmedical”), and this also occurs in queries: Test query 2251 contains “sustainedvowel”. Such tokens cannot match their component words. Display equations are absent from the extracted text altogether, and the `abstract` field contains more merged words than the copy of the abstract that appears as each paper’s first paragraph (Section 2.2).

Table 1: Paragraph collection statistics under three preprocessing settings, defined in Section 2.1.

	Baseline	Cleaned	+ Snowball
Total tokens	167,965,275	114,907,087	114,907,087
Unique terms	781,405	659,861	589,251
Terms appearing once	367,415	297,672	272,983
Median length	58	40	40
90th pct. length	143	104	104
Longest paragraph	32,070	20,818	20,818
Over 1,000 tokens	868	117	117
Empty paragraphs	429	8,389	8,389
Under 10 tokens	82,390	230,782	230,782

Table 1 compares three settings. *Baseline* applies only lowercasing, tokenization on letters, digits, and underscores, and stopword removal (with “not” and “no” retained). *Cleaned* applies steps 1–5 and no longer retains “not” and “no”: bag-of-words models cannot use them to express negation, and “not” alone accounted for over 500,000 tokens. *+ Snowball* is the final pipeline. Cleaning reduced the collection from 168.0 million to 114.9 million tokens, a 31.6% decrease, and all of the 50 most frequent tokens became words, such as “model”, “data”, and “using”. It also reduced the number of paragraphs longer than 1,000 tokens from 868 to 117; the longest Baseline paragraphs were collaboration author lists made up largely of LaTeX markup and ORCID identifiers. Stemming leaves the number of tokens unchanged but shrinks the vocabulary by a further 10.7%, from 659,861 to 589,251 terms. The cost is that some distinct words share a stem: “different” and “difference” both become “differ”, and “generate”, “generation”, and “generative” all become “generat”. Table 1 also shows that the number of paragraphs under 10 tokens nearly triples (82,390 → 230,782). This is because the cleaning process employed in the latter two pipelines strips math out of math-heavy paragraphs and leaves only their prose, causing many to become short fragments. Finally, cleaning removes every token from 8,389 paragraphs. Of these, 94% contain inline math or LaTeX commands: they are standalone formulas, sometimes with a few connecting words (“if”, “then”, “where”) that are themselves stopwords. We describe how these paragraphs are handled in Section 2.2.

2.2 Document Representation

2.3 TF-IDF

2.4 BM25

BM25 addresses two weaknesses of the TF-IDF weighting in Section 2.3: it saturates the contribution of repeated terms, and it normalizes document length relative to the collection average rather than by vector length. We describe the scoring function we use, our implementation, and how we tuned its two parameters, k_1 and b, for each collection.

2.4.1 Scoring Function. BM25 scores a document d for a query q as

$$\text{score}(q,d) = \sum_{t \in q} idf_t \cdot \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\big(1 - b + b\,L_d/L_{avg}\big)} \tag{1}$$

$$idf_t = \ln\left(1 + \frac{N - df_t + 0.5}{df_t + 0.5}\right) \tag{2}$$

where the sum runs over query tokens, so a term repeated in the query contributes once per occurrence. The term-frequency component saturates: as $tf_{t,d}$ grows, it approaches $k_1 + 1$, so additional occurrences of a term add progressively less. The parameter k_1 controls how quickly this happens; a small k_1 makes the score depend mainly on whether the term occurs at all, while a large k_1 lets repetition keep contributing. The parameter $b \in [0, 1]$ controls length normalization: a document longer than average ($L_d > L_{avg}$) has a larger denominator and therefore a lower score for the same term frequency, with $b = 0$ disabling normalization and $b = 1$ applying it fully.

We use the “Lucene (accurate)” variant of BM25 described by Kamphuis et al. [2], the third row of their Table 1. Since they found no significant differences in effectiveness among eight BM25 variants, we chose on practical grounds. First, its IDF adds one inside the logarithm. The original formulation [4] assigns negative weight to terms that occur in more than half the documents, so matching such a term would lower a document’s score, and a document matching more query terms could score lower than one matching fewer. In our paper collection, 312 terms exceed this threshold, including words common in our natural-language queries such as “use”, “result”, and “differ” (as in “How does X differ from Y?”). Second, it uses exact document lengths, which our index already stores, rather than Lucene’s compressed approximation, which Kamphuis et al. found gave negligible speed benefits. Third, variants designed for long documents, such as BM25L and BM25+, add a further parameter to tune without significant gains in their study. Following Robertson et al. [4], we also multiply the term-frequency component by $k_1 + 1$; this scales all scores by a constant and does not change the ranking.

2.4.2 Implementation. Our implementation scores only documents that contain at least one query term, accumulating each term’s contribution over its postings (term-at-a-time). Two quantities are precomputed: idf_t for every term when the model is created, and the length-normalization term $k_1(1-b+b\,L_d/L_{avg})$ for every document, which is recomputed only when k_1 or b change. A term repeated in the query is processed once, with its contribution multiplied by its query frequency. After scoring, the top 100 documents are selected, with ties broken by internal document number so that runs are reproducible.

Our first implementation accumulated scores in a Python loop over individual postings. It was correct, but averaged 91 ms per query on papers and 1.08 s per query on paragraphs, so a full paragraph tuning grid (48 parameter settings for 1,600 queries) would have required roughly a day of computation. We therefore vectorized scoring with NumPy, so that only the loop over query terms remains in Python. Each term’s postings (its document numbers

Table 2: nDCG@5 on 400 held-out Train queries not used for tuning.

	Selected (k_1, b)	Tuned	Textbook $(1.2, 0.75)$	Default $(0.9, 0.4)$
Papers	$(4.0, 0.75)$	**0.473**	0.454	0.436
Paragraphs	$(0.4, 0.9)$	**0.371**	0.354	0.344

and term frequencies) are converted to arrays once and cached; the term's contributions to all documents in its postings are then computed in one array operation and added to a dense score array indexed by document number. The top 100 are found by partial selection rather than a full sort. This reduced average search time to 15 ms on papers and 105 ms on paragraphs, speedups of 6× and 10× (Section 3.2), and the full paper grid search ran in about eight minutes. The vectorized version produced the exact same evaluation metrics as the original on our tuning queries over the full index.

2.4.3 Parameter Tuning. We tuned k_1 and b separately for each collection on the Train queries, never using Study queries. All 2,000 Train queries were shuffled with a fixed seed and split into 1,600 tuning queries and 400 held-out queries. On the tuning queries, we evaluated every setting in a grid: $k_1 \in \{0.9, 1.2, 1.8, 2.4, 3, 4, 5, 6, 8\}$ for papers and $k_1 \in \{0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 1.5, 1.8\}$ for paragraphs, each with $b \in \{0.3, 0.5, 0.6, 0.75, 0.9, 1.0\}$. We selected the setting with the highest nDCG@5. Among our evaluation metrics, nDCG@5 is the only one that both uses graded relevance and focuses on the top of the ranking, which is what users of the interface see first; deeper metrics such as P@10 are less informative here, because most queries have only a few judged documents.

The selected settings are $k_1 = 4.0$, $b = 0.75$ for papers and $k_1 = 0.4$, $b = 0.9$ for paragraphs (Figures 1 and 2). The tenfold difference in k_1 likely reflects document length. Papers average about 3,350 tokens, so a term repeated many times is genuinely informative about a paper's topic, and a large k_1 lets repetition keep contributing. Paragraphs have a median length of about 40 tokens and rarely repeat a term, so a small k_1, which makes the score depend more on whether a term occurs, works best. Both collections favor substantial length normalization: $b = 0.3$ is the worst value in every row of the paper grid. The paper optimum also lies on a broad plateau ($k_1 \geq 3$, $0.5 \leq b \leq 0.9$) in which scores differ by about 0.01 or less, so the exact choice within it matters little.

On the 400 held-out queries, the tuned settings outperformed both the common textbook values ($k_1 = 1.2$, $b = 0.75$) and the default of the Anserini toolkit used by Kamphuis et al. ($k_1 = 0.9$, $b = 0.4$) for both collections (Table 2), so the selected parameters generalize beyond the queries used to choose them. The choice is also robust to the target metric: tuning for nDCG or MRR instead selected the samesetting for papers, and tuning for MRR on a separate sample of 1,000 Train queries selected the same setting for paragraphs. We compare tuned and default parameters on the Study queries in Section 4.

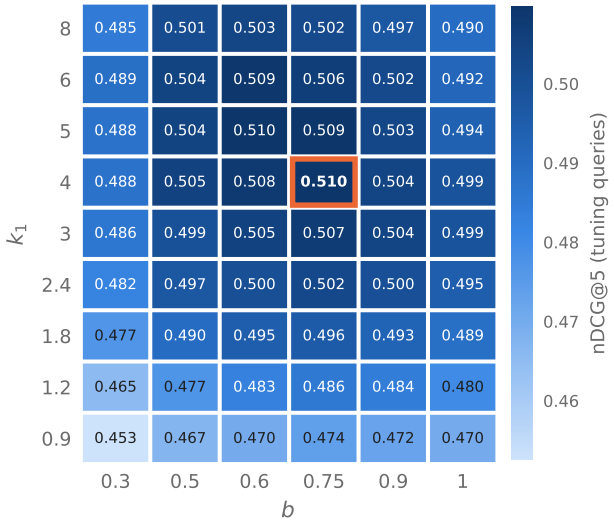


Figure 1: BM25 nDCG@5 on 1,600 Train tuning queries (paper collection). Rows and columns are the tested values, not to scale; scores are rounded to three decimals. The outlined cell is the selected setting ($k_1 = 4.0$, $b = 0.75$), which scored highest before rounding. It lies within a broad plateau ($k_1 \geq 3$, $0.5 \leq b \leq 0.9$) in which settings differ by about 0.01 or less.

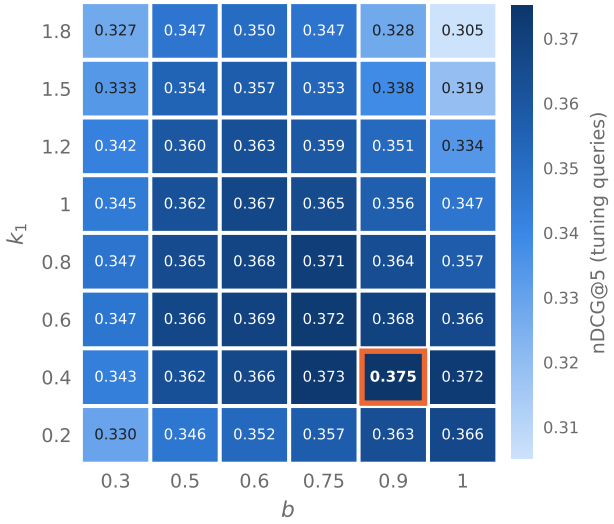


Figure 2: BM25 nDCG@5 on 1,600 Train tuning queries (paragraph collection), formatted as in Figure 1. The outlined cell is the selected setting ($k_1 = 0.4$, $b = 0.9$). The region within about 0.01 of it is narrower than for papers ($0.4 \leq k_1 \leq 0.6$, $0.75 \leq b \leq 1.0$), and scores fall off sharply for large k_1 combined with strong length normalization.

2.5 Evaluation Setup

2.6 Search Interface

3 Evaluation Results

3.1 Metrics

3.2 Efficiency

Table 3: Indexing, startup, and query times, and index sizes. Query times are averages over the 250 Study queries (pure-Python BM25: 50 queries) and exclude query tokenization; startup includes loading the index and each model's precomputation. Measured single-threaded on a shared server, so times are approximate.

	Papers	Paragraphs
Documents	34,308	2,267,185
Avg. document length (tokens)	3,361	50.7
Indexing (build and save)	66 s	182 s
Index size on disk (JSON)	242 MB	974 MB
Startup, BM25	9.4 s	28.7 s
Startup, TF-IDF	18.1 s	68.7 s
Query, BM25 (NumPy)	9.2 ms	74.3 ms
Query, BM25 (pure Python)	91 ms	1,076 ms
Query, TF-IDF (pure Python)	95.6 ms	1,115.5 ms

All times were measured single-threaded with Python 3.8 on a shared server with an Intel Xeon Gold 5215 CPU (2.50 GHz) and 62 GiB of RAM. Preprocessing the 34,308 raw papers into both collections takes about 4.5 minutes, after which building both indexes takes about 4 minutes (Table 3).

Building the paragraph index takes about three times as long as the paper index, although both collections contain a similar number of tokens, because the paragraph index records many more postings: each term is stored once per paragraph in which it occurs rather than once per paper. The same effect governs query time. Paragraph queries take longer than paper queries despite much shorter documents, because the postings of a common query term span far more documents (up to 2.27 million rather than 34 thousand), and every posting must be scored.

The gap between BM25 and TF-IDF reflects our implementations rather than the models themselves. TF-IDF uses the same pure-Python loop over postings as our original BM25, and the two run at nearly the same speed (95.6 vs. 91 ms on papers; 1,115.5 vs. 1,076 ms on paragraphs), so vectorizing TF-IDF would likely yield a similar speedup of roughly an order of magnitude. TF-IDF also has a longer startup, since it computes a vector norm for every document by walking all postings before the first query; for paragraphs this adds about 40 seconds.

Our indexes are stored as JSON, which is simple but costly to load: the 974 MB paragraph index occupies about 5.5 GB of memory during retrieval, because each posting is stored as Python objects, and loading accounts for most of the startup time. Storing postings as compact numeric arrays would reduce both memory use and startup time.

4 Results Analysis

Table 4: Preprocessing ablation on Study queries, nDCG@5, with BM25 parameters fixed to the values tuned for the main configuration (math removed, Snowball stemming). Comparing the first two columns isolates math removal; comparing the last two isolates stemming. * marks a significant difference from the column to its left (paired t-test, $p < 0.05$).

	Main	Math kept	Math kept, no stemming
Papers, TF-IDF	0.313	0.320	0.379*
Papers, BM25	0.514	0.534*	0.542
Paragraphs, TF-IDF	0.229	0.238	0.235
Paragraphs, BM25	0.416	0.416	0.378*

We isolated two preprocessing choices on Study queries (Table 4). Removing inline math, which reduced the collection by 31.6%, had little effect on retrieval. It made no significant difference for either model on paragraphs or for TF-IDF on papers, and slightly reduced paper BM25 (nDCG@5 0.534 to 0.514, significant only for nDCG@5 and strict P@5). Papers contain enough spelled-out notation, such as "epsilon" or "kappa", for kept math tokens to provide a few additional correct matches. Math removal is a reasonable tradeoff for a 31.6% reduction in collection size.

Stemming mattered more, in opposite directions for our two strongest effects. It significantly improved paragraph BM25 on every metric (nDCG@5 0.378 to 0.416, MRR 0.457 to 0.509), but significantly reduced paper TF-IDF on every metric (nDCG@5 0.379 to 0.320); it had no significant effect on paper BM25 or paragraph TF-IDF. A plausible explanation is that short paragraphs gain from the additional matches stemming creates, while for long papers under cosine normalization, conflated stems (e.g., "generat" for "generate", "generation", and "generative") make common stems even more frequent and dilute term weights. Regardless, it is important to note that even TF-IDF's best configuration (no stemming, 0.379 nDCG@5) stays far below BM25 (0.542). This implies that the reasons for BM25's better performance over TF-IDF do not necessarily originate from the preprocessing pipeline.

5 Conclusion

References

[1] Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural Language Processing with Python.* O'Reilly Media.

[2] Chris Kamphuis, Arjen P. de Vries, Leonid Boytsov, and Jimmy Lin. 2020. Which BM25 Do You Mean? A Large-Scale Reproducibility Study of Scoring Variants. In *Advances in Information Retrieval: 42nd European Conference on IR Research (ECIR 2020).* Springer, 28–34.

[3] Martin F. Porter. 2001. Snowball: A Language for Stemming Algorithms. https://snowballstem.org/texts/introduction.html.

[4] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline M. Hancock-Beaulieu, and Mike Gatford. 1994. Okapi at TREC-3. In *Proceedings of the Third Text REtrieval Conference (TREC-3).* 109–126.